

1. HTML in Python

1) Introduction to embedding HTML within Python using web frameworks like Django or Flask

Ans-

Concept:

- HTML is used for structure and design of web pages.
- Python (through frameworks like Django or Flask) handles the backend logic — such as fetching data, processing input, etc.
- HTML is embedded or connected to Python using **templates**.

In Django:

- Django uses a **templates directory** for HTML files.
- Data is passed from **views.py** to HTML using a **context dictionary**.
- Example:
 - # views.py
 - from django.shortcuts import render
 -
 - def home(request):
 - return render(request, 'index.html', {'name': 'Mitesh'})

```
<!-- index.html -->
```

```
<h1>Welcome, {{ name }}</h1>
```

2) Generating dynamic HTML content using Django templates.

Ans-

Django uses a **template system** to generate dynamic HTML pages. Templates allow you to insert data from Python (backend) into HTML pages dynamically.

Steps:

1. Create a view in views.py:

- This function prepares data and sends it to the HTML template.
 - `from django.shortcuts import render`
 - `def student_info(request):`
 - `context = {`
 - `'name': 'Mitesh',`
 - `'course': 'Python Backend Development',`
 - `'marks': 88`
 - `}`
 - `return render(request, 'student.html', context)`

2. Create a template (student.html):

- Use Django template syntax (`{{ }}`) to display variables.
 - `<html>`
 - `<body>`
 - `<h2>Student Information</h2>`
 - `<p>Name: {{ name }}</p>`
 - `<p>Course: {{ course }}</p>`
 - `<p>Marks: {{ marks }}</p>`
 - `</body>`
 - `</html>`

3. Add a URL in urls.py:

- `from django.urls import path`
- `from . import views`
-
- `urlpatterns = [`
- `path('student/', views.student_info, name='student_info'),`
- `]`

2. CSS in Python

1) Integrating CSS with Django templates

Ans-

To style your Django web pages, you link **CSS files** to your HTML templates. Django handles CSS and other files (like images or JavaScript) as **static files**.

Steps:

1. Create a static folder:

Inside your app directory, create a folder named **static**

Example:

```
myapp/
├── static/
│   └── style.css
├── templates/
│   └── index.html
└── views.py
```

2. Write your CSS file (style.css):

```
body {
    background-color: #f4f4f4;
    font-family: Arial, sans-serif;
}

h1 {
    color: blue;
    text-align: center;
}
```

3. Configure static settings in settings.py:

Make sure you have these lines:

```
STATIC_URL = '/static/'
```

4. Load and link CSS in your HTML template (index.html):

```
{% load static %}

<html>

<head>

    <link rel="stylesheet" href="{% static 'style.css' %}">

</head>

<body>

    <h1>Welcome to Django</h1>

</body>

</html>
```

2) How to serve static files (like CSS, JavaScript) in Django.

Ans-

Static files are assets like CSS, JavaScript, and images used to style and add functionality to web pages. Django provides a built-in way to manage and serve them.

Steps:

1. **Create a static folder:**

Inside your Django app, create a folder named **static**.

```
myapp/
├── static/
│   ├── style.css
│   └── script.js
├── templates/
│   └── index.html
└── views.py
```

2. Add static settings in settings.py:

```
STATIC_URL = '/static/'
```

3. Use static files in your HTML template:

- Load the static tag.
- Link CSS and JS files using `{% static %}`.

```
{% load static %}
```

```
<html>
```

```
<head>
```

```
<link rel="stylesheet" href="{% static 'style.css' %}">
```

```
<script src="{% static 'script.js' %}"></script>
```

```
</head>
```

```
<body>
```

```
<h1>Hello Django</h1>
```

```
</body>
```

```
</html>
```

4. Run the development server:

When `DEBUG = True`, Django automatically serves static files.

```
python manage.py runserver
```

3. JavaScript with Python

1) Using JavaScript for client-side interactivity in Django templates.

Ans-

1. Create a static folder for JavaScript:

```
myapp/
```

```
├── static/
```

```
|   └── script.js
```

```
└── templates/
```

| └─ index.html

|─ views.py

2. Write your JavaScript code (script.js):

```
function showMessage() {  
    alert("Welcome to Django with JavaScript!");  
}
```

3. Load and use JavaScript in your template (index.html):

```
{% load static %}  
<html>  
<head>  
    <script src="{% static 'script.js' %}"></script>  
</head>  
<body>  
    <h2>JavaScript in Django Template</h2>  
    <button onclick="showMessage()">Click Me</button>  
</body>  
</html>
```

4. Run your Django server:

```
python manage.py runserver
```

2) Linking external or internal JavaScript files in Django.

Ans-

1. Internal JavaScript:

Save JS file in the static folder.

Example:

```
{% load static %}  
  
<script src="{% static 'script.js' %}"></script>
```

2. External JavaScript:

Use a direct link (like a CDN).

Example:

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

4. Django Introduction

Q) Overview of Django: Web development framework.

Ans-

Django is a high-level **Python web framework** that allows developers to build secure, scalable, and maintainable web applications quickly.

Key points:

- **MVC Architecture:** Uses MTV (Model-Template-View) pattern for clean code separation.
- **Built-in features:** Authentication, ORM (Object Relational Mapping), admin interface, routing, forms, and security tools.
- **Template system:** Allows dynamic HTML generation.
- **Static files management:** Handles CSS, JavaScript, and images.
- **Scalable and secure:** Designed for rapid development with strong security practices.

Q) Advantages of Django (e.g., scalability, security)

Ans-

1. **Scalability:** Handles high-traffic websites efficiently.
2. **Security:** Built-in protection against SQL injection, cross-site scripting, and other threats.
3. **Rapid Development:** Includes ready-to-use features like authentication, admin panel, and ORM.
4. **Versatility:** Suitable for various projects — from simple websites to complex applications.
5. **Reusability:** Modular structure allows reuse of code and apps.
6. **Community Support:** Large community and extensive documentation.
7. **Maintainability:** Follows the DRY (Don't Repeat Yourself) principle for clean code.

Q) Django vs. Flask comparison: Which to choose and why

Ans-

Django vs. Flask Comparison

Feature	Django	Flask
Type	Full-stack web framework	Micro web framework
Learning Curve	Steeper due to many built-in tools	Easier and simpler for beginners
Built-in Features	Authentication, admin panel, ORM, forms	Minimal — you add features yourself
Scalability	Highly scalable	Scalable but requires more setup
Flexibility	Less flexible (follows strict structure)	More flexible, you choose structure
Development Speed	Faster for large projects with built-in tools	Slower for large projects (more manual work)
Best For	Large, complex projects needing structure	Small projects, APIs, or prototypes

Which to choose and why:

- Choose **Django** if you want a full-featured framework for large projects with rapid development and built-in tools.
- Choose **Flask** if you want more flexibility, control, and a lightweight framework for small projects or APIs.

5. Virtual Environment

Q) Understanding the importance of a virtual environment in Python projects

Ans-

A **virtual environment** is an isolated workspace for a Python project. It keeps dependencies separate from the global Python installation.

Key benefits:

1. **Avoid conflicts:** Different projects can use different versions of packages without interference.
2. **Project isolation:** Keeps dependencies and settings separate for each project.

3. **Reproducibility:** Makes it easier to share projects with others using the same dependencies.
4. **Clean environment:** Prevents cluttering the global Python installation.

Q) Using venv or virtualenv to create isolated environments.

Ans-

Using venv or virtualenv to Create Isolated Environments

A virtual environment keeps project dependencies separate from the global Python installation.

1. Using venv (built-in in Python 3.3+):

- Create environment:

Activate environment:

- Windows:
 - `env\Scripts\activate`

Deactivate environment:

`Deactivate`

6. Project and App Creation

Q) Steps to create a Django project and individual apps within the project

Ans-

Steps to Create a Django Project and Individual Apps

1. **Install Django** (if not installed):

`pip install django`

2. Create a Django Project:

`django-admin startproject project_name`

This creates a folder with the project structure:

`project_name/`

```
manage.py
project_name/
  __init__.py
  settings.py
  urls.py
  asgi.py
  wsgi.py
```

3. Navigate to Project Folder:

```
cd project_name
```

4. Create a Django App:

```
python manage.py startapp app_name
```

```
app_name/
  __init__.py
  admin.py
  apps.py
  models.py
  tests.py
  views.py
  migrations/
```

5. Register the App:

- Open `settings.py` in the project folder.
- Add `'app_name'`, to the `INSTALLED_APPS` list.

Q) Understanding the role of `manage.py`, `urls.py`, and `views.py`.

Ans-

manage.py

- A command-line utility for interacting with your Django project.
- Used for running the development server, applying migrations, creating apps, etc.
- Example commands:

```
python manage.py runserver
```

```
python manage.py migrate
```

```
python manage.py startapp app_name
```

urls.py

- Defines URL patterns for your project or app.
- Maps URLs to the corresponding view functions.
- Example:

```
from django.urls import path

from . import views

urlpatterns = [

    path("", views.home, name='home'),

]
```

views.py

- Contains functions or classes that handle requests and return responses.
- Connects the backend logic with templates.
- Example:

```
from django.shortcuts import render

def home(request):

    return render(request, 'home.html')
```

7. MVT Pattern Architecture

Q) Django's MVT (Model-View-Template) architecture and how it handles request-response cycles.

Ans-

MVT Architecture:

- **Model:** Handles data and database operations (defines structure of data using Django ORM).
- **View:** Contains logic to process requests and return responses (usually by rendering templates).
- **Template:** HTML files that define how data is presented to the user.

How Django Handles Request-Response Cycle:

Step-by-Step:

1. **Request:** User sends a request (e.g., opening a URL in the browser).
2. **URL Routing (urls.py):** Django matches the request URL with URL patterns defined in urls.py.
3. **View Function (views.py):** The matched URL calls the corresponding view function.
4. **Model Interaction:** The view may interact with the model to fetch or save data.
5. **Template Rendering:** View sends data to a template for HTML generation.
6. **Response:** The rendered HTML is sent back to the user's browser.

Example:

User request → urls.py → views.py → models.py (optional) → template → HTTP Response

8. Django Admin Panel

Q) Introduction to Django's built-in admin panel

Ans-

The Django **admin panel** is a pre-built web interface that allows developers and administrators to manage database records easily without writing SQL queries.

Key points:

1. **Automatic creation:** Django automatically provides the admin interface after creating a project.
2. **Access:** Run the server and go to
<http://127.0.0.1:8000/admin/>
3. Superuser creation:
python manage.py createsuperuser
Enter a username, email, and password to create an admin account
4. Register models:
To manage models in the admin panel, register them in admin.py:
from django.contrib import admin
from .models import Book

admin.site.register(Book)
5. Features:
 - Add, edit, and delete data.
 - Search and filter records.
 - Manage users and permissions.

Q) Customizing the Django admin interface to manage database records.

Ans-

Register the model in admin.py:

```
from django.contrib import admin  
from .models import Book
```

```
@admin.register(Book)
```

```
class BookAdmin(admin.ModelAdmin):
```

```
    list_display = ('title', 'author', 'published_date')    # Columns shown in list view  
    search_fields = ('title', 'author')                    # Adds search box  
    list_filter = ('published_date',)                      # Adds filter sidebar
```

```
ordering = ('title',) # Sorts records
```

Run the server and open:

```
http://127.0.0.1:8000/admin/
```

You can now:

- View records in a customized table.
- Search and filter data easily.
- Sort and manage records directly from the admin panel.

9. URL Patterns and Template Integration

Q) Setting up URL patterns in urls.py for routing requests to views.

Ans-

Create a view in views.py:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to the Home Page")
```

Add URL pattern in the app's urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Include app URLs in the project's main urls.py:

```
from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('myapp.urls')),

]
```

Q) Integrating templates with views to render dynamic HTML content.

Ans-

In Django, templates are used to create dynamic web pages by combining HTML with data passed from views. To integrate templates with views:

Steps:

1. Create a Template

- Inside your app, create a folder named templates, and within it, create an HTML file.

```
myapp/
  └─ templates/
      └─ home.html
```

Example (home.html):

```
<html>

<body>

    <h1>Welcome, {{ user_name }}!</h1>

</body>

</html>
```

2. Configure Templates in settings.py

- Make sure Django knows where to find templates:

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'], # global templates folder  
        'APP_DIRS': True,  
    },  
]
```

3. Render Template in View

- Use Django's render() function to pass data to the template.

```
from django.shortcuts import render
```

```
def home_view(request):  
    context = {'user_name': 'Mitesh'}  
    return render(request, 'home.html', context)
```

4. Map the View to a URL

- Add a URL pattern in urls.py.

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.home_view, name='home'),  
]
```

10. Form Validation using JavaScript

Q) Using JavaScript for front-end form validation

Ans-

Front-end form validation using **JavaScript** ensures that user inputs are correct before sending data to the server. It helps improve user experience and reduce invalid data submissions.

```
<!DOCTYPE html>

<html>

<head>

  <title>Form Validation</title>

  <script>

    function validateForm() {

      let name = document.forms["myForm"]["username"].value;

      let email = document.forms["myForm"]["email"].value;

      if (name === "") {

        alert("Name must be filled out");

        return false;

      }

      if (!email.includes("@")) {

        alert("Please enter a valid email address");

        return false;

      }

      return true; // Form will submit if all checks pass

    }

  </script>

</head>

<body>

  <form name="myForm" onsubmit="return validateForm()">
```

```
Name: <input type="text" name="username"><br><br>
Email: <input type="text" name="email"><br><br>
<input type="submit" value="Submit">
</form>
</body>
</html>
```

11. Django Database Connectivity (MySQL or SQLite)

Q) Connecting Django to a database (SQLite or MySQL).

Ans-

Django connects to databases through settings defined in the settings.py file under the DATABASES section.

1. SQLite (Default)

SQLite is the default database for Django projects.

Example (default setup in settings.py):

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

2. MySQL

To use MySQL, first install the connector:

```
pip install mysqlclient
```

Then, update settings.py:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'your_database_name',  
        'USER': 'your_username',  
        'PASSWORD': 'your_password',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

3. Apply Migrations

After configuration, run:

```
python manage.py makemigrations  
python manage.py migrate
```

Q) Using the Django ORM for database queries.

Ans-

Django ORM (Object-Relational Mapper) allows you to interact with the database using Python code instead of SQL queries.

1. Creating a Model

In models.py:

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.IntegerField()
```

2. Making and Applying Migrations

```
python manage.py makemigrations  
python manage.py migrate
```

3. Performing Queries

```
# Import model  
from .models import Student  
  
# Create record  
Student.objects.create(name="Mitesh", age=20)  
  
# Read records  
students = Student.objects.all()  
  
# Filter records  
Student.objects.filter(age=20)  
  
# Update record  
Student.objects.filter(name="Mitesh").update(age=21)  
  
# Delete record  
Student.objects.filter(name="Mitesh").delete()
```

12. ORM and QuerySets

Q) Understanding Django's ORM and how QuerySets are used to interact with the database.

Ans-

Answer:

Django's ORM (Object-Relational Mapper) lets you interact with the database using Python objects instead of SQL queries. Each model class represents a table, and its objects represent rows in that table.

1. What is a QuerySet?

A **QuerySet** is a collection of database records retrieved through a Django model. It allows filtering, sorting, and retrieving data efficiently.

2.Example:

```
from .models import Student

# Get all records
students = Student.objects.all()

# Filter records
young_students = Student.objects.filter(age__lt=18)

# Get a single record
student = Student.objects.get(id=1)

# Order records
ordered_students = Student.objects.order_by('name')
```

3. QuerySet Features:

- **Lazy loading:** QuerySets are not executed until needed.
- **Chaining:** You can combine filters (e.g., `.filter().order_by()`).
- **Return type:** QuerySets return model objects, not raw SQL results.

13. Django Forms and Authentication

Q) Using Django's built-in form handling.

Ans-

Django provides a **forms framework** to create, validate, and process HTML forms easily.

1. Create a Form

In forms.py:

```
from django import forms

class StudentForm(forms.Form):
    name = forms.CharField(max_length=100)
    age = forms.IntegerField()
```

2. Create a View to Handle the Form

In views.py:

```
from django.shortcuts import render
from .forms import StudentForm

def student_view(request):
    if request.method == "POST":
        form = StudentForm(request.POST)
        if form.is_valid():
            name = form.cleaned_data['name']
            age = form.cleaned_data['age']
            # Save or process data
        else:
            form = StudentForm()

    return render(request, 'student_form.html', {'form': form})
```

3. Create a Template for the Form

In student_form.html:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

4. Map URL to View

In urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('student/', views.student_view, name='student_form'),
]
```

Q) Implementing Django's authentication system (sign up, login, logout, password management)

Ans-

Django provides a built-in authentication system that handles user registration, login, logout, and password management.

1. Sign Up (User Registration)

Create a form using Django's built-in UserCreationForm.

views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth.forms import UserCreationForm
```

```
def signup_view(request):
    if request.method == "POST":
        form = UserCreationForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('login')
    else:
        form = UserCreationForm()
    return render(request, 'signup.html', {'form': form})
```

signup.html

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Sign Up</button>
</form>
```

2. Login

Use Django's built-in AuthenticationForm.

views.py

```
from django.contrib.auth import authenticate, login
from django.contrib.auth.forms import AuthenticationForm

def login_view(request):
    if request.method == "POST":
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
```



```

        username = form.cleaned_data.get('username')
        password = form.cleaned_data.get('password')
        user = authenticate(username=username, password=password)
        if user:
            login(request, user)
            return redirect('home')
        else:
            form = AuthenticationForm()
        return render(request, 'login.html', {'form': form})

```

login.html

```

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Login</button>
</form>

```

3. Logout

views.py

```

from django.contrib.auth import logout

def logout_view(request):
    logout(request)
    return redirect('login')

```

urls.py

```

from django.urls import path

from . import views

```

```
urlpatterns = [
    path('signup/', views.signup_view, name='signup'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
]
```

4. Password Management

Django provides built-in views for:

- **Password change:** `django.contrib.auth.views.PasswordChangeView`
- **Password reset:** `django.contrib.auth.views.PasswordResetView`

```
from django.contrib.auth import views as auth_views
```

```
urlpatterns += [
    path('password_change/', auth_views.PasswordChangeView.as_view(),
         name='password_change'),
    path('password_reset/', auth_views.PasswordResetView.as_view(),
         name='password_reset'),
]
```

14. CRUD Operations using AJAX

Q) Using AJAX for making asynchronous requests to the server without reloading the page.

Ans-

AJAX (Asynchronous JavaScript and XML) allows you to send and receive data from the server **without refreshing the page**, improving user experience.

1. Create a View in Django

views.py

```
from django.http import JsonResponse
```

```
def ajax_view(request):  
    data = {"message": "Hello from Django!"}  
    return JsonResponse(data)
```

2. Set Up URL

urls.py

```
from django.urls import path  
  
from . import views  
  
urlpatterns = [  
    path('ajax/', views.ajax_view, name='ajax_view'),  
]
```

3. HTML Template with JavaScript

template.html

```
<html>  
  
<head>  
  
    <script>  
        function fetchData() {  
            fetch("/ajax/")  
                .then(response => response.json())  
                .then(data => {  
                    document.getElementById("result").innerText = data.message;  
                });  
        }  
    </script>  
</head>
```

```

<body>

    <button onclick="fetchData()">Click to Fetch Data</button>

    <p id="result"></p>

</body>

</html>

```

4. How it Works

- User clicks the button → JavaScript sends a request to /ajax/.
- Django view processes the request and returns JSON.
- JavaScript updates the HTML without reloading the page.

15. Customizing the Django Admin Panel

Q) Techniques for customizing the Django admin panel

Ans-

Django's admin panel can be customized to improve usability and match project requirements.

1. Register Models with Custom Admin Classes

In admin.py:

```

from django.contrib import admin

from .models import Book

class BookAdmin(admin.ModelAdmin):

    @admin.register(Book)

    list_display = ('title', 'author', 'published_date') # Columns shown
    search_fields = ('title', 'author')                  # Search box
    list_filter = ('published_date',)                     # Filter sidebar

```

```
ordering = ('title',)
```

```
# Default ordering
```

2. Customize Form Layout

Use fields or fieldsets:

```
class BookAdmin(admin.ModelAdmin):  
    fieldsets = (  
        (None, {'fields': ('title', 'author')}),  
        ('Advanced options', {'fields': ('published_date',), 'classes': ('collapse',)}),  
    )
```

3. Add Actions

Define actions to perform bulk changes

```
def make_published(modeladmin, request, queryset):  
    queryset.update(status='published')  
  
class BookAdmin(admin.ModelAdmin):  
    actions = [make_published]
```

4. Add Inline Models

Display related models inline:

```
from .models import Chapter  
  
class ChapterInline(admin.TabularInline):  
    model = Chapter  
    extra = 1  
  
class BookAdmin(admin.ModelAdmin):  
    inlines = [ChapterInline]
```

5. Customize Admin Site

In admin.py:

```
admin.site.site_header = "Library Admin"

admin.site.site_title = "Library Management"

admin.site.index_title = "Welcome to Library Admin"
```

16. Payment Integration Using Paytm

Q) Introduction to integrating payment gateways (like Paytm) in Django projects.

Ans-

Integrating a payment gateway like **Paytm** in a Django project allows your application to process online payments securely and efficiently. This is useful for e-commerce sites, subscription services, or any project requiring payment transactions.

Key Steps in Integration:

1. Create a Merchant Account

- Register at Paytm's developer portal.
- Obtain credentials: **Merchant ID**, **Merchant Key**, and **Website Name**.

2. Install Required Libraries

- For Paytm, install the official checksum library:
- `pip install paytmchecksum`

3. Configure Django Settings

Store Paytm credentials in settings.py:

```
PAYTM_MERCHANT_KEY = "your_merchant_key"

PAYTM_MERCHANT_ID = "your_merchant_id"

PAYTM_WEBSITE = "WEBSTAGING"

PAYTM_CHANNEL_ID = "WEB"
```

PAYTM_INDUSTRY_TYPE_ID = "Retail"

PAYTM_URL = <https://securegw-stage.paytm.in/order/process>

4 Create **Payment Request View**

- Prepare payment parameters and generate a checksum for security.

5 Redirect **User to Paytm Payment Page**

- Pass parameters via HTML form to Paytm's payment URL.

6 Handle **Payment Response**

- Paytm sends a callback to your defined URL after payment.
- Verify the checksum and payment status before processing.

17. GitHub Project Deployment

Q) Steps to push a Django project to GitHub

Ans-

Steps to push a Django project to GitHub:

1. **Create a GitHub Repository**

- Go to [GitHub](#)
- Give it a name and click **Create repository**.

2. **Initialize Git in Project Folder**

Open terminal in your Django project folder:

git init

3. **Create .gitignore File**

Add a .gitignore file to exclude unnecessary files:

```
venv/  
__pycache__/  
*.pyc  
db.sqlite3  
*.log  
.env
```

4. **Add Files to Git**

git add .

5.Commit Changes

```
git commit -m "Initial commit"
```

6. Add Remote Repository

```
git remote add origin https://github.com/yourusername/your-repo.git
```

7. Push Project to GitHub

```
git branch -M main
```

```
git push -u origin main
```

18. Live Project Deployment (PythonAnywhere)

Q) Introduction to deploying Django projects to live servers like PythonAnywhere.

Ans-

Deploying a Django project means making it available on the internet so users can access it through a live domain. **PythonAnywhere** is a popular hosting platform for deploying Django projects easily.

Steps to Deploy on PythonAnywhere:

1. Create an Account

- Go to PythonAnywhere
- Free and paid plans are available.

2.Upload Your Project

- Use the PythonAnywhere dashboard to upload your Django project.
- You can upload via **Git**, **SCP**, or by uploading a ZIP file and extracting it.

3.Set Up a Virtual Environment

In PythonAnywhere's Bash console:

```
mkvirtualenv myenv --python=python3.x
```

```
pip install django
```

```
pip install -r requirements.txt
```


4. Configure Django Settings for Deployment

- In settings.py:
 - Set DEBUG = False.
 - Add your domain to ALLOWED_HOSTS.
 - Configure STATIC_ROOT and MEDIA_ROOT

5. Collect Static Files

```
python manage.py collectstatic
```

6. Configure Web App on PythonAnywhere

- Go to the **Web** tab.
- Add a new web app and select Django.
- Enter your project path and virtual environment path.

7. Set Up WSGI File

- Edit the WSGI configuration file to point to your Django project.

8. Run Database Migrations

```
python manage.py migrate
```

9. Reload Web App

- On the PythonAnywhere dashboard, reload your web app.

19. Social Authentication

Q) Setting up social login options (Google, Facebook, GitHub) in Django using OAuth2.

Ans-

Social login allows users to sign in using accounts from services like Google, Facebook, or GitHub. In Django, this can be implemented using the **django-allauth** library.

1. Install django-allauth

```
pip install django-allauth
```

2. Add to INSTALLED_APPS in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.sites',  
    'allauth',  
    'allauth.account',  
    'allauth.socialaccount',  
    'allauth.socialaccount.providers.google',  
    'allauth.socialaccount.providers.facebook',  
    'allauth.socialaccount.providers.github',  
]
```

```
SITE_ID = 1
```

3. Update urls.py

```
from django.urls import path, include  
  
urlpatterns = [  
    path('accounts/', include('allauth.urls')),  
]
```

4. Configure Authentication Backends in settings.py

```
AUTHENTICATION_BACKENDS = [  
    'django.contrib.auth.backends.ModelBackend',  
    'allauth.account.auth_backends.AuthenticationBackend',  
]
```

5. Add Social App Credentials

- Go to provider's developer console (Google, Facebook, GitHub).
- Create an app and get **Client ID** and **Client Secret**.
- In Django Admin → Social Applications → Add application for each provider.
- Select provider, add credentials, and choose the site.

6.Run Migrations

```
python manage.py migrate
```

7. Use Login Link

Example template:

```
<a href="{% provider_login_url 'google' %}">Login with Google</a>
<a href="{% provider_login_url 'facebook' %}">Login with Facebook</a>
<a href="{% provider_login_url 'github' %}">Login with GitHub</a>
```

20. SMS Sending APIs (Twilio)

Q) Introduction to Twilio API for sending SMS and OTPs

Ans-

Twilio is a cloud communication platform that allows developers to send SMS, OTPs, and make calls using APIs. In Django, it can be integrated to send text messages directly from your application.

1. Install Twilio Library

```
pip install twilio
```

2.Sign up on Twilio

- Go to <https://www.twilio.com>

3. Set up Django View to Send SMS

```
from twilio.rest import Client
```

```
def send_sms():
```

```
    account_sid = 'your_account_sid'
```

```
    auth_token = 'your_auth_token'
```

```
    client = Client(account_sid, auth_token)
```

```
    message = client.messages.create(
```

```
        body="Your OTP is 123456",
```

```

from_='+1234567890', # Twilio number

to='+919876543210' # User's number

)

print(message.sid)

```

4. Use it for OTP Verification

- Generate a random OTP in Django using Python's random module.
- Send the OTP via Twilio SMS.
- Ask the user to enter the OTP and verify it on the server side.

21. Payment Integration (PayPal, Stripe)

Q) Introduction to integrating payment gateways like PayPal and Stripe

Ans-

Payment gateways like **PayPal** and **Stripe** are used to process online payments securely in Django applications. They allow users to pay using credit/debit cards or online wallets without storing sensitive information on your server.

1. Install Required Libraries

- For PayPal:

```
pip install django-paypal
```

For Stripe:

```
pip install stripe
```

2. Install Required Libraries

```

INSTALLED_APPS = [

    'paypal.standard.ipn', # For PayPal

    # or

    'stripe',

]

```

3. Configure API Keys

- Get **Client ID** and **Secret Key** from the PayPal/Stripe dashboard.
- Add them in settings.py:
 - STRIPE_PUBLIC_KEY = 'your_public_key'
 - STRIPE_SECRET_KEY = 'your_secret_key'

4.Create Views and Templates

- Set up a payment view that uses the API to create a payment session or redirect users to the payment page.

5. Handle Payment Success and Failure

- Add callback or webhook URLs to update payment status after completion.

22. Google Maps API Integration

Q) Using Google Maps API to display maps and calculate distances between locations.

Ans-

The **Google Maps API** allows Django applications to display interactive maps and calculate distances between two points (like user location and destination). It is commonly used in travel, delivery, and location-based apps.

1.Get Google Maps API Key

Go to Google Cloud Console

Enable **Maps JavaScript API** and **Distance Matrix API**.

Copy your **API key**.

2.Display Map in Template

In your Django template (HTML):

```
<div id="map" style="height:400px;width:100%;"></div>
```

```
<script>
```

```
function initMap() {
```

```
    var location = {lat: 23.0225, lng: 72.5714}; // Example: Ahmedabad
```

```
    var map = new google.maps.Map(document.getElementById('map'), {
```

```
        zoom: 10,
```

```
        center: location
    });

    new google.maps.Marker({position: location, map: map});
}
</script>
```

```
<script
src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&callback=initM
ap" async defer></script>
```

3.Calculate Distance (Using Distance Matrix API)

import requests

```
def calculate_distance(origin, destination):

    api_key = "YOUR_API_KEY"

    url =
f"https://maps.googleapis.com/maps/api/distancematrix/json?origins={origin}&destinations
={destination}&key={api_key}"

    response = requests.get(url).json()

    distance = response['rows'][0]['elements'][0]['distance']['text']

    return distance
```